

SDEV2171

Lesson 13: Loading States and Error Handling

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Lesson Objectives

By the end of this lesson, students should be able to:

- Explain why loading states matter in async screens
- Add a visible loading indicator while a request is in progress
- Catch request failures and store an error state
- Show a user-friendly error message and retry path
- Refactor the lesson-12 fetch screen to represent request states clearly

WE ARE ESSENTIAL TO ALBERTA | NAIT

Lesson Roadmap

1. Reconnect lesson-12 fetch logic to visible request states
2. Define loading, success, and error as part of one request lifecycle
3. Set up the local backend testing path
4. Guided build of one resilient request screen
5. Testing and troubleshooting
6. Checkpoint and lesson-14 handoff

WE ARE ESSENTIAL TO ALBERTA | NAIT

Starting Point From Lesson 12

- keep the same local backend data shape from the earlier examples: `{ id, label }`
- keep the same two-screen Expo Router flow
- confirm the lesson-12 fetch screen still works before adding more state

If your project is not ready

- if your baseline is `resolved`, keep using your own project
- if your baseline is `partial` or `blocked`, recover the lesson-12 fetch baseline first
- if recovery will take too long, switch to the instructor-guided reference path and record your next step

WE ARE ESSENTIAL TO ALBERTA | NAIT

Why Blank Screens Feel Broken

- a network request takes time
- if the screen shows nothing while waiting, the user cannot tell whether the app is working
- if the request fails silently, the user cannot recover

Plain-language rule

- every request should tell the user what state it is in

WE ARE ESSENTIAL TO ALBERTA | NAIT

The Request Lifecycle Today

1. loading
2. success
3. error

What changes from lesson 12

- the request syntax can stay almost the same
- the screen now needs extra UI rules around that request

WE ARE ESSENTIAL TO ALBERTA | NAIT

Three State Questions

- What should the user see while the request is still running?
- What should the user see when the request succeeds?
- What should the user see when the request fails?

WE ARE ESSENTIAL TO ALBERTA | NAIT

Local Backend Path

- start the backend server in a separate terminal
- keep the same `study-steps` endpoint shape from the example flow
- use the backend `delay` parameter to make loading visible
- use the backend `error=1` parameter to trigger the error branch on purpose

WE ARE ESSENTIAL TO ALBERTA | NAIT

Minimal New State

```
const [remoteSteps, setRemoteSteps] = useState([]);
const [isLoading, setIsLoading] = useState(true);
const [errorMessage, setErrorMessage] = useState('');
```

What each value means

- `remoteSteps` : the success data
- `isLoading` : whether the request is still in progress
- `errorMessage` : the current failure message, if one exists

WE ARE ESSENTIAL TO ALBERTA | NAIT

Loading-State Rule

```
setIsLoading(true);
setErrorMessage('');
```

When to do this

- before the request starts
- before a retry starts

WE ARE ESSENTIAL TO ALBERTA | NAIT

Error-State Rule

```
catch (error) {
  setErrorMessage(error.message);
}
```

Important idea

- the error is not only a terminal log
- the screen should store it and show the user something meaningful

WE ARE ESSENTIAL TO ALBERTA | NAIT

Finish-State Rule

```
finally {
  setIsLoading(false);
}
```

Why this matters

- the loading state should end whether the request succeeded or failed

WE ARE ESSENTIAL TO ALBERTA | NAIT

One Conditional Rendering Pattern

```
if (isLoading) {
  return <LoadingCard />;
}

if (errorMessage) {
  return <ErrorCard />;
}

return <SuccessList />;
```

WE ARE ESSENTIAL TO ALBERTA | NAIT

Guided Build Order

1. confirm the lesson-12 success path still works
2. confirm the local backend is running
3. add `isLoading` and `errorMessage` state
4. set loading before the request starts
5. clear old errors before each new request
6. store the failure message in `catch`
7. stop loading in `finally`
8. add visible loading, success, and error branches
9. test the error path on purpose
10. test retry and recovery

WE ARE ESSENTIAL TO ALBERTA | NAIT

What To Keep From Lesson 12

- the same local backend endpoint shape
- the same `useEffect` trigger for first load
- the same list rendering pattern
- the same back button

What to add

- one loading indicator
- one error message branch
- one retry path

WE ARE ESSENTIAL TO ALBERTA | NAIT

Wrong State Vs Right State

Wrong

- loading is only implied by a blank screen
- errors only print in the terminal
- the user has no retry action

Right

- the loading state is visible
- the error state is visible
- the user can try again without restarting the app

WE ARE ESSENTIAL TO ALBERTA | NAIT

One Full Example Loop

Loading test

- request `/study-steps?delay=1200`
- confirm the indicator appears before the list

Error test

- request `/study-steps?delay=1200&error=1`
- confirm the error message branch appears

Recovery test

- retry the normal request
- confirm the success list returns

WE ARE ESSENTIAL TO ALBERTA | NAIT

Mid-Build Checkpoint

Students should be ready to show

- one visible loading branch
- one visible error branch
- one success branch using the same study-step data
- one retry or reload action

Students should also be able to answer

- Why does a loading state help the user?
- Where is the error message stored?
- What makes the loading state stop in both success and failure cases?

WE ARE ESSENTIAL TO ALBERTA | NAIT

What We Learned Today

- async requests need visible request-state feedback
- loading, success, and error are all part of one lifecycle
- `try`, `catch`, and `finally` support that lifecycle
- conditional rendering turns request state into user-visible UI
- local backend controls can make testing more reliable than an external demo API

WE ARE ESSENTIAL TO ALBERTA | NAIT

Lesson 14 Bridge

- next lesson can use this stronger request-state pattern in a larger applied workflow
- keep the same habits:
  - make state visible
  - test success and failure on purpose
  - keep recovery paths clear

WE ARE ESSENTIAL TO ALBERTA | NAIT

Next Step

- keep the local backend flow working in a known-good state
- keep the request-state branches readable
- be ready to use the same patterns in the next lesson

WE ARE ESSENTIAL TO ALBERTA | NAIT